

LocalBuka Intern Assessment

Track: Backend Engineering

Time Limit:	3-5 Days (Take-Home)
Target Level:	Software Engineering Intern (Calibrated Scope)
Submission:	Email your public GitHub repository link and a 2-minute walkthrough video to hellohr@localbuka.com

Product Context

LocalBuka connects customers to local food vendors (bukas) in Nigeria. For the Backend Intern track, you will build a clean, type-safe REST API using NestJS (or Node.js/Express) and TypeScript. You do not need to build any user interface. Focus entirely on clean routing, database queries, logic validation, and error handling.

Practical Requirements

You will implement 4 core REST endpoints based on our provided schema and mock data:

- **GET /restaurants:** List all restaurants. Implement basic search by restaurant name. If query params latitude and longitude are provided, return the restaurants sorted by distance (closest first).
- **GET /restaurants/:id:** Fetch detailed info on a single restaurant, including its reviews.
- **POST /restaurants/:id/reviews:** Add a new review (rating scale 1-5). Ensure a user can only review the same restaurant once.
- **POST /points/earn:** Awards points for a check-in (50 pts) or review (20 pts). Restrict check-ins to once per user per restaurant per calendar day.
- **GET /points/balance/:userId:** Fetch current total points and points transaction ledger.

Technical Assessment Questions

Q1. TypeScript Data Shape Parsing

Write a parser function `parseRestaurant(raw: unknown): Restaurant` in TypeScript. It must cast raw incoming snake_case fields or string numbers into correct types, throwing a validation error if required fields are missing.

Q2. Handling Race Conditions

Describe how you would prevent race conditions if a user sends multiple concurrent points redemption requests at the exact same millisecond. Propose one database level lock strategy.

Evaluation Rubric

Your submission will be scored out of 100 points based on the following breakdown:

Criteria	Weight	Description
API Endpoint Functionality	35 pts	All endpoints work, return exact JSON schemas, and handle status codes correctly.
Business Logic & Constraints	25 pts	Correctly blocks duplicate reviews and restricts check-ins to once per day.
Database & Geolocation Math	20 pts	Calculates distances using coordinates and orders list correctly.
TypeScript & Code Hygiene	20 pts	Strict typings used, proper NestJS modules, no raw 'any' types.